

BitTorrent WebSeed Migration

Proof of Concept

HTTP/3 File Download with Transparent QUIC Server Migration

PoC Documentation • July 2026

This document describes a proof-of-concept implementation demonstrating QUIC server-side migration applied to the BitTorrent WebSeed (BEP 19) use case. A primary HTTP/3 server begins serving a large file download, then transparently migrates the connection to a preferred server on a different machine. The client receives the complete file without interruption, and verifies integrity via SHA-256 hash comparison. This PoC validates that QUIC server migration is directly applicable to HTTP/3-based file distribution systems like WebSeed, requiring **no protocol changes** to existing BitTorrent clients.

1. Overview

BitTorrent WebSeed (BEP 19) allows .torrent files to include standard HTTP URLs as backup download sources alongside traditional BitTorrent peers. When the swarm has insufficient seeders, clients fall back to these HTTP-based sources to download pieces of the file.

Modern WebSeed servers increasingly support HTTP/3, which runs over QUIC. This creates a natural fit for QUIC server-side migration: the WebSeed server can transparently migrate an active file download from one machine to another without the client needing to re-establish the connection or re-request the data.

Key Concepts

- **BEP 19 (WebSeed):** HTTP/HTTPS URLs embedded in .torrent files as fallback download sources
- **HTTP/3 over QUIC:** Modern WebSeeds use HTTP/3, which provides the QUIC transport layer
- **Server Migration:** QUIC's preferred_address mechanism (RFC 9000 Section 9.6) allows the primary server to redirect the client to a preferred server mid-connection
- **This PoC:** Two HTTP/3 servers host the same large file; the primary migrates to the preferred server mid-download, demonstrating seamless failover

Advantage: WebSeed migration requires zero changes to the BitTorrent protocol or client software -- it operates entirely at the QUIC transport layer.

2. Architecture

The PoC uses the existing four-machine testbed on the same LAN (141.217.168.x). Three machines participate directly in the WebSeed migration scenario:

Role	Machine	IP Address	Description
Primary WebSeed	opti7040	141.217.168.152	Serves file via HTTP/3, advertises preferred address, exports migration state
Preferred WebSeed	homeserver2	141.217.168.143	Imports migration state, handles PATH_CHALLENGE, continues serving file
Client	optiplex7010	141.217.168.127	Downloads file via HTTP/3, verifies SHA-256 integrity after migration

Migration Flow

1. Client connects to Primary WebSeed (opti7040:4433) via HTTP/3
2. Primary completes QUIC handshake, advertising preferred_address = 141.217.168.143:4433
3. Primary begins serving the requested file, computing SHA-256 hash
4. Primary exports migration state (~445 bytes: TLS secrets, CIDs, packet numbers)
5. Migration state is transferred to Preferred server via configured backend (TCP/Redis/HTTP)
6. Client sends PATH_CHALLENGE to preferred address (141.217.168.143:4433)
7. Preferred server imports crypto state, decrypts PATH_CHALLENGE, sends PATH_RESPONSE

- 8. Client validates PATH_RESPONSE -- migration complete, file download continues from preferred server

3. Implementation Details

webseed-primary Binary

The primary server binary (webseed-primary) is a specialized HTTP/3 server built on top of the modified Neco QUIC stack. It reads a file from disk, computes its SHA-256 hash using a pure Rust implementation, and serves it over HTTP/3 with metadata headers for client-side integrity verification.

HTTP/3 Response Headers

Header	Value	Purpose
content-type	application/octet-stream	Binary file download
content-disposition	attachment; filename="<name>"	Suggests filename to the client
content-length	<size in bytes>	Total file size for progress tracking
x-sha256	<hex-encoded hash>	SHA-256 of file contents for client-side integrity verification

The x-sha256 header is critical: it allows the client to verify that the file received after migration is byte-identical to what the primary server intended to serve. The client computes SHA-256 of the downloaded file and compares it against this header value.

Preferred Server

The preferred server reuses the existing preferred-server binary from the core migration implementation. It imports the migration state (TLS secrets, connection IDs, packet numbers), handles the PATH_CHALLENGE/PATH_RESPONSE exchange with the client, and can continue serving data on the migrated connection.

SHA-256 Implementation

File hashing uses a pure Rust SHA-256 implementation to avoid external dependencies. The hash is computed once when the file is loaded into memory, then served in the response header for every request. This ensures consistent integrity verification regardless of which server ultimately delivers the file content.

4. How to Run

Step 1: Create a Test File

Generate a 100MB random binary file to use as the download payload:

```
dd if=/dev/urandom of=/tmp/test_100mb.bin bs=1M count=100
```

Step 2: Start the Preferred Server (homeserver2)

```
preferred-server 141.217.168.143:4433 9999
```

The preferred server listens on port 4433 (QUIC) and port 9999 (state transfer). It waits for migration state from the primary before accepting client connections.

Step 3: Start the Primary WebSeed Server (opti7040)

```
webseed-primary 0.0.0.0:4433 141.217.168.143:4433 \  
/tmp/test_100mb.bin 141.217.168.143:9999
```

Argument	Example	Description
Listen address	0.0.0.0:4433	Address and port for incoming QUIC connections
Preferred address	141.217.168.143:4433	Address advertised in preferred_address transport parameter
File path	/tmp/test_100mb.bin	Path to the file to serve via HTTP/3
State transfer target	141.217.168.143:9999	Preferred server's state transfer endpoint

Step 4: Download the File

Use any HTTP/3-capable client to download the file from the primary server. The QUIC migration happens transparently during the download. After completion, compare the SHA-256 hash of the downloaded file against the x-sha256 response header to verify integrity.

5. .torrent Integration

The PoC includes a `create_torrent.sh` script that generates a `.torrent` file with both servers listed as WebSeeds. This demonstrates how the migration mechanism integrates with the standard BitTorrent ecosystem.

url-list Field

BEP 19 defines the `url-list` field in `.torrent` metadata. This field contains one or more HTTP/HTTPS URLs that serve the same file content. BitTorrent clients that support WebSeed (including aria2, qBittorrent,

Transmission) will use these URLs as additional download sources.

```
url-list:  
- https://141.217.168.152:4433/test_100mb.bin  
- https://141.217.168.143:4433/test_100mb.bin
```

Client Compatibility

aria2 is the recommended client for testing, as it supports both HTTP/3 (via nghttp3) and BitTorrent WebSeed. When aria2 connects to the primary WebSeed URL over HTTP/3, the QUIC server migration occurs transparently at the transport layer. aria2 sees a normal HTTP/3 download completing successfully.

6. Test Scenarios

Test	Description	Success Criteria
Integrity Test	Download 100MB file through migration, compute SHA-256 of received file	SHA-256 of downloaded file matches x-sha256 header from primary server
Failover Test	Primary server fails mid-download, preferred server continues serving	Download completes without error after primary goes offline
Latency Comparison	Measure recovery time: QUIC migration vs. client discovering backup WebSeed	Migration recovery is faster than client-side WebSeed failover (no new handshake)
Piece Verification	Download via .torrent file, verify all pieces pass BitTorrent hash check	All pieces pass hash verification in the BitTorrent client

Integrity Test Details

The integrity test is the core validation of the PoC. The primary server computes SHA-256 of the file at startup and includes it in the x-sha256 response header. After the download completes (potentially through migration), the client independently computes SHA-256 of the received file and compares the two values. A match confirms that the migration did not corrupt any data.

Latency Comparison Details

Without QUIC migration, a WebSeed failover requires the client to: (1) detect the primary server is down (timeout), (2) resolve the backup WebSeed URL, (3) perform a new TLS/QUIC handshake, and (4) resume the download from the last known offset. With QUIC migration, the PATH_CHALLENGE/PATH_RESPONSE exchange completes in a single RTT, and the connection continues with the same TLS session -- no new handshake required.

Expected result: QUIC migration recovery completes in ~1 RTT (~0.5ms on LAN), while traditional WebSeed failover takes 10-30 seconds (timeout + new handshake).

7. BitTorrent Relevance

This PoC validates QUIC server migration in the context of BitTorrent file distribution. The following table summarizes the applicability landscape:

Scenario	Transport	Migration Applicability	Status
WebSeed (BEP 19)	HTTP/3 (QUIC)	Directly applicable today -- no protocol changes needed	This PoC
Seeder-to-Seeder	BitTorrent (TCP/uTP)	Would require QUIC transport for BitTorrent (does not exist yet)	Future work
Tracker Failover	HTTP/HTTPS	Tracker announces could use HTTP/3, enabling migration	Possible
DHT Node Migration	UDP	DHT uses raw UDP, not QUIC; would need protocol redesign	Not applicable

Why WebSeed is the Right Target

- **No protocol changes:** WebSeed already uses HTTP, and modern HTTP uses QUIC. Server migration operates at the QUIC transport layer, invisible to the application.
- **Real-world deployment:** Many CDNs and file hosting services serve as WebSeeds. QUIC migration enables seamless failover between CDN edge nodes.
- **Immediate benefit:** Unlike seeder-to-seeder migration (which would require a QUIC-based BitTorrent protocol), WebSeed migration works with existing clients today.
- **Integrity preserved:** BitTorrent's piece hashing and the x-sha256 header provide independent verification that migration did not corrupt data.

Regular seeder-to-seeder migration would require QUIC as a transport for the BitTorrent protocol itself. Since BitTorrent currently uses TCP and uTP (a UDP-based protocol unrelated to QUIC), this is not feasible without significant protocol changes. WebSeed migration sidesteps this entirely by operating at the HTTP/3 layer.